

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
X = [1, 2, 3, 4, 5, None, 7, 8, 9, 10, 11, 12, None, 14, 15, 16, 17, 18, 19, 20]
y_reg = [2, 4, 6, 8, 10, None, 14, 16, 18, 20, 22, 24, None, 28, 30, 32, 34, 36, 38, 40]
y_cls = [0, 0, 0, 0, 0, None, 0, 1, 1, 1, 1, 1, None, 1, 1, 1, 1, 1, 1, 1]
```

```
# df=pd.DataFrame(X,columns=['X'])
```

```
# df
```

```
# df1=pd.DataFrame(y_reg,columns=['y_reg'])
```

```
df=pd.DataFrame({'x':X , 'y_reg':y_reg , 'y_cls':y_cls})
```

```
df
```

	x	y_reg	y_cls
0	1.0	2.0	0.0
1	2.0	4.0	0.0
2	3.0	6.0	0.0
3	4.0	8.0	0.0
4	5.0	10.0	0.0
5	NaN	NaN	NaN
6	7.0	14.0	0.0
7	8.0	16.0	1.0
8	9.0	18.0	1.0
9	10.0	20.0	1.0
10	11.0	22.0	1.0
11	12.0	24.0	1.0
12	NaN	NaN	NaN
13	14.0	28.0	1.0
14	15.0	30.0	1.0
15	16.0	32.0	1.0
16	17.0	34.0	1.0
17	18.0	36.0	1.0
18	19.0	38.0	1.0
19	20.0	40.0	1.0

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df.isnull().sum()
```

```

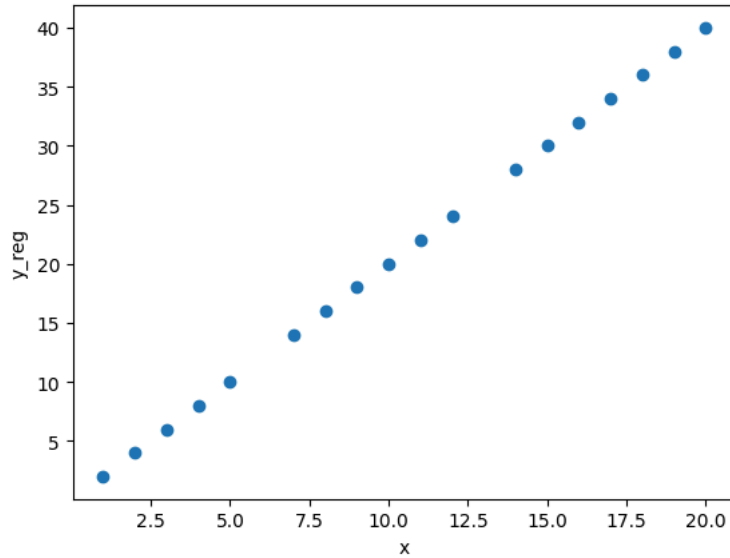
      0
x      2
y_reg  2
y_cls  2

dtype: int64
```

```
df=df.dropna()
```

```
plt.scatter(df['x'],df['y_reg'])
plt.xlabel("x")
plt.ylabel("y_reg")
```

Text(0, 0.5, 'y_reg')



```
x=df['x'].values.reshape(-1,1)
y=df['y_reg']
```

```
from sklearn.model_selection import train_test_split
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y)
```

```
from sklearn.linear_model import LinearRegression
```

```
model=LinearRegression()
```

```
model.fit(x , y)
```

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

```
y_pred = model.predict([[7]])
display(y_pred)
```

```
array([14.])
```

```
from sklearn.linear_model import LogisticRegression
```

```
lr=LogisticRegression()
lr.fit(x,y)
```

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

▼ LogisticRegression ⓘ ?

```
LogisticRegression()
```

```
y_lr=lr.predict([[3]])
y_lr
```

```
array([4.])
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier(n_neighbors=3)
```

```
knn.fit(x,y)
```

▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier(n_neighbors=3)

```
knn.predict([[5]])
```

```
array([6.])
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf = RandomForestClassifier(n_estimators=10, random_state=42)
```

```
rf.fit(x_train,y_train)
```

▼ RandomForestClassifier ⓘ ?
RandomForestClassifier(n_estimators=10, random_state=42)

```
rf.predict([[5]])
```

```
array([6.])
```

Start coding or [generate](#) with AI.